

**Practical File  
Of  
Software Engineering Laboratory  
(UGCA1924)**



**Submitted By: -**

Bikkar Singh  
Roll No. 2447072  
Class: BCA 4th Sem

**Submitted to: -**

Ms. Sharndeeep Kaur  
Assistant professor  
(Dept. of Computer Application)

**PUNJAB INSTITUTE OF MANAGEMENT AND TECHNOLOGY  
(AFFILIATED TO IKG PUNJAB TECHNICAL UNIVERSITY, JALANDHAR)**

### Practical File Index Format

| Sr. No. | Name of the Experiment/Program                                                                                           | Page No. | Teacher's Signature |
|---------|--------------------------------------------------------------------------------------------------------------------------|----------|---------------------|
| 1       | Identify project scope and objective of given problem:<br>a. College automation system.<br>b. Banking Management System. | 3        |                     |
| 2       | Develop software requirements specification for (1.a) and (1.b) problem.                                                 | 5        |                     |
| 3       | Develop UML Use case model for a problem.                                                                                | 7        |                     |
| 4       | Develop Class diagrams.                                                                                                  | 8        |                     |
| 5       | Represent project Scheduling of above-mentioned projects.                                                                | 9        |                     |
| 6       | Use any model for estimating the effort, schedule and cost of software project.                                          | 10       |                     |
| 7       | Develop DFD model (level-0, level-1 DFD and Data dictionary) of the project.                                             | 11       |                     |
| 8       | Develop sequence diagram.                                                                                                | 13       |                     |
| 9       | Develop Structured design for the DFD model developed.                                                                   | 14       |                     |
| 10      | Develop the waterfall model, prototype model and spiral model of the product.                                            | 15       |                     |
| 11      | Explain with reason which model is best suited for the product.                                                          | 17       |                     |
| 12      | Develop a working protocol of any of two problem.                                                                        | 18       |                     |
| 13      | Use LOC, FP and Cyclomatic Complexity Metric of above-mentioned problem.                                                 | 19       |                     |
| 14      | Find Maintainability Index and Reliability Index of above-mentioned problem.                                             | 21       |                     |
| 15      | Using any Case Tool find number of statements, depth and complexity of the prototype.                                    | 22       |                     |

## **Practical 01: Identify project scope and objective of given problem:**

### **1. College automation system.**

### **2. Banking Management System.**

#### **1). College automation system**

##### ***Introduction:***

- This section introduces Unity College website project. It also contains definitions, acronyms and abbreviations used throughout the document .
- It will allow college to address potential students by showcasing their course and facilities and its current students and faculty by providing access to the various facilities provided by the college in an easy and interactive manner.
- Moreover, it will help people in college and outside of it to know about news and events happening in college.

##### ***Scope:***

- Development of a centralized platform for managing student records, faculty details, courses, and administrative activities.
- Automation of tasks such as admission processes, fee payments, attendance, exam scheduling, and result generation.
- Integration of features like notifications, academic calendars, and communication systems.

##### ***Objective:***

- To streamline administrative and academic processes for efficiency and transparency.
- To reduce manual errors and workload for both students and staff.
- To provide a user-friendly interface for all stakeholders, including students, faculty, and administration.

#### **2). Banking Management System**

##### ***Introduction:***

- BANK MANAGEMENT SYSTEM undertaken as a project is based on relevant technologies.
- The main aim of this project is to develop software for bank management system.
- This project is to develop software for bank management system. This project has been developed to carry out the processes easily and quickly, which is not possible with the manual systems, which are overcome by this software.
- This project is developed using VB language and. Hence it provides the complete solution for the current management system.

**Scope:**

- Creation of a robust system for managing customer accounts, transactions, loans, and other financial services.
- Implementation of security measures for online and offline transactions.
- Integration of features such as account tracking, report generation, and customer service.

**Objective:**

- To enhance the accuracy and efficiency of banking operations.
- To ensure customer satisfaction with secure and seamless banking experiences.
- To facilitate data-driven decision-making through comprehensive reports and insights.

## **Practical 02: Develop software requirements specification for (1 a.) and (1 b.) problem.**

### **(1 a) College Automation System**

#### ***Introduction***

- **Purpose:** To automate academic and administrative tasks for a college, ensuring efficiency, transparency, and convenience.
- **Scope:** Covers processes such as admissions, fee payments, attendance tracking, result generation, and communication.
- **Audience:** College administrators, faculty, and students.

#### ***Functional Requirements***

- **User Authentication:** Separate login portals for administrators, faculty, and students.
- **Admission Management:** Online application submission, status tracking, and fee payment.
- **Attendance System:** Online attendance marking and monthly reports.
- **Examination & Results:** Exam scheduling, grade entry, and report card generation.
- **Notifications:** Real-time communication for updates, schedules, and announcements.

#### ***Non-Functional Requirements***

- **Performance:** System should support simultaneous users without delays.
- **Security:** Role-based access control and secure data handling.
- **Usability:** Intuitive interface for all users.
- **Compatibility:** Accessible on multiple devices (PCs, tablets, and smartphones).

#### ***Constraints***

- System must comply with relevant educational and data protection regulations.
- Should function with minimal downtime.

#### ***Assumptions***

- All users will have internet access.
- The institution will provide necessary hardware.

### **(1 b) Banking Management System**

#### ***Introduction***

- **Purpose:** To manage customer accounts and financial transactions securely and efficiently.
- **Scope:** Includes account creation, transaction handling, loan management, and report generation.
- **Audience:** Bank employees, customers, and administrators.

#### ***Functional Requirements***

- **User Authentication:** Secure login for bank staff and customers.

- **Account Management:** Opening and closing accounts, balance inquiries, and updating details.
- **Transaction Processing:** Deposits, withdrawals, and fund transfers.
- **Loan Management:** Loan application, approval, and repayment tracking.

#### ***Non-Functional Requirements***

- **Performance:** Capable of handling high transaction volumes.
- **Security:** Robust encryption for data storage and communication.
- **Usability:** Designed for both tech-savvy users and beginners.

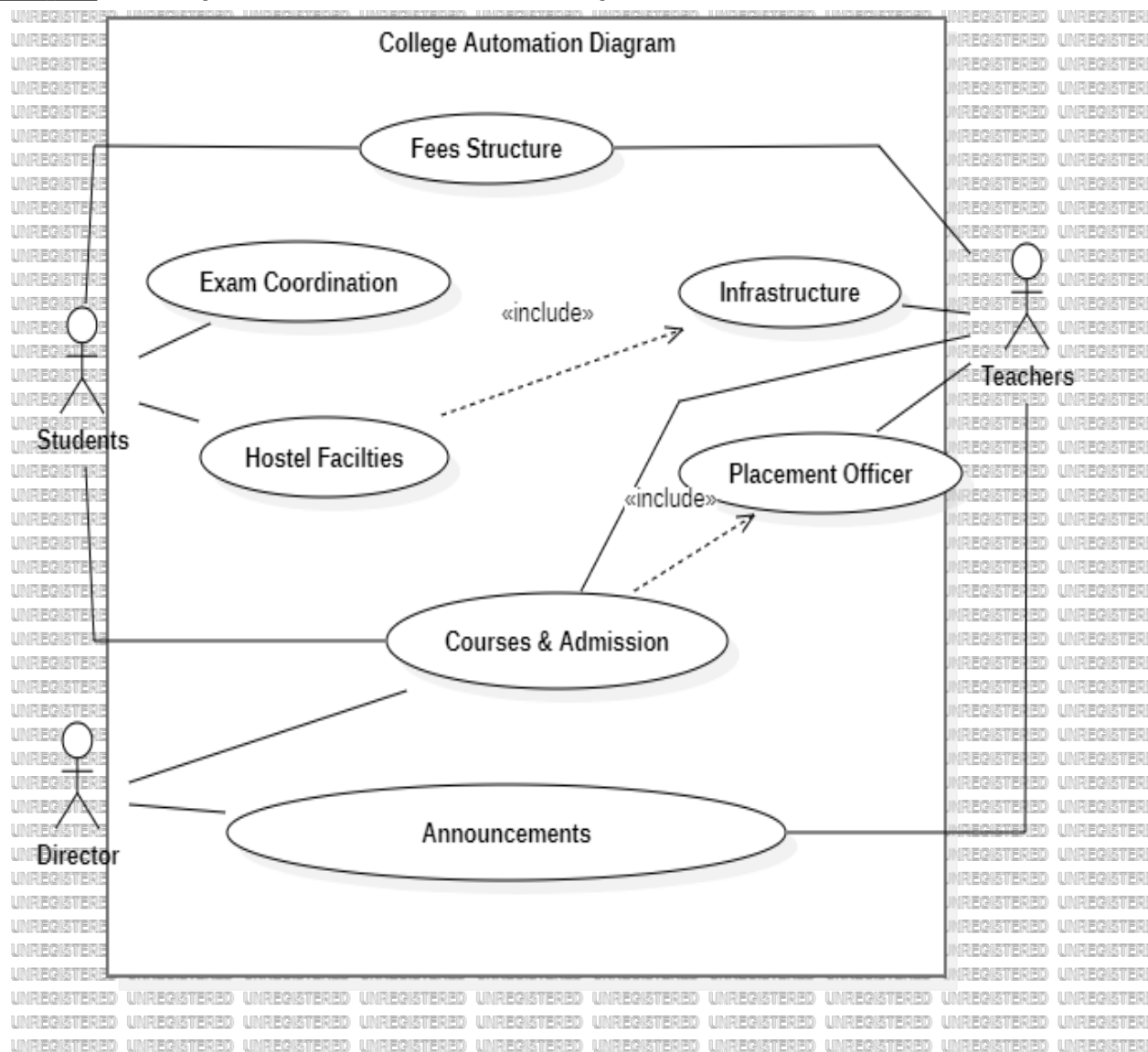
#### ***Constraints***

- Must comply with banking regulations and security standards.
- Must support both online and offline operations.

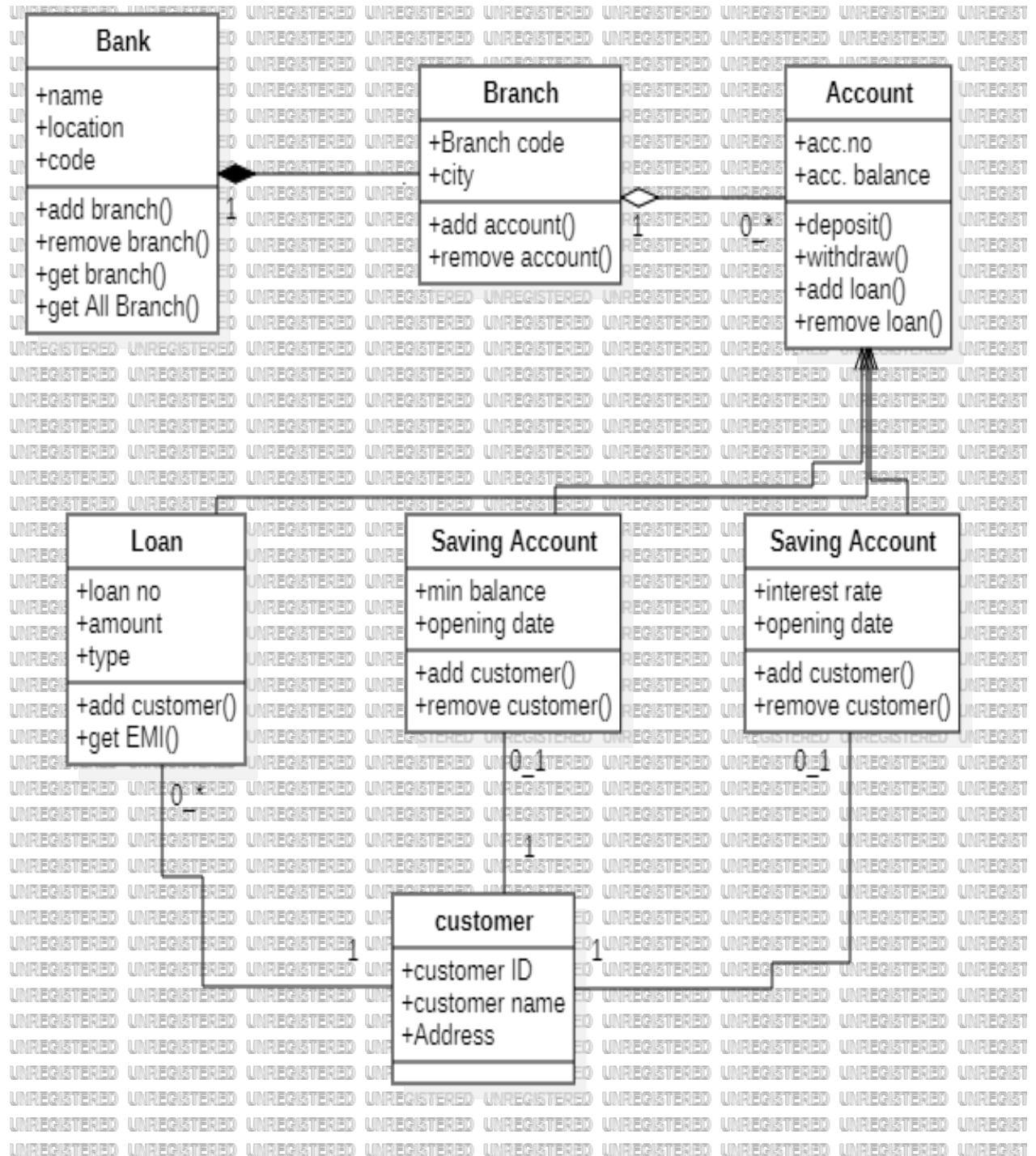
#### ***Assumptions***

- Customers will have basic computer knowledge.
- Bank infrastructure will support the new system.

### Practical 03: Develop UML Use case model for a problem.

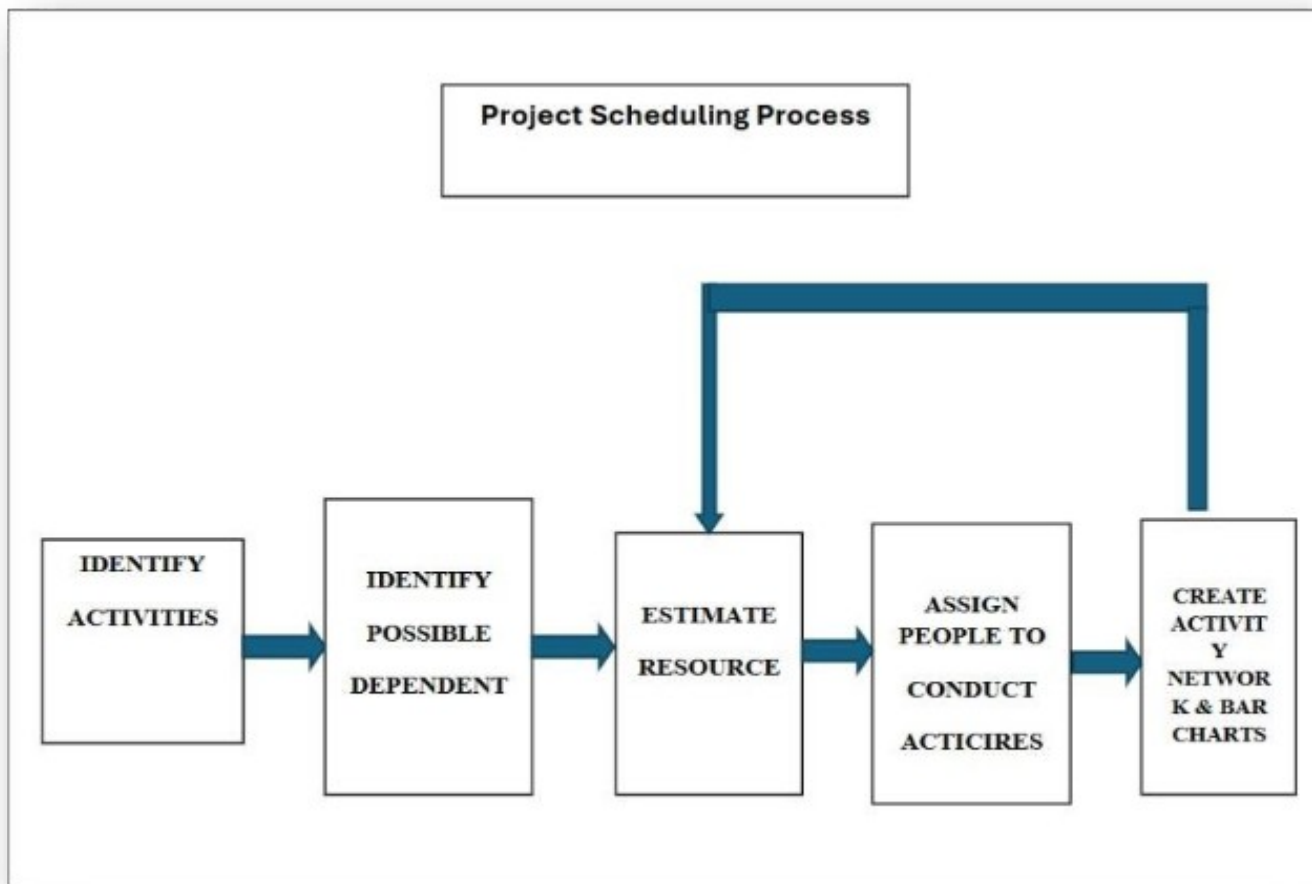


## Practical 04: Develop Class diagrams.

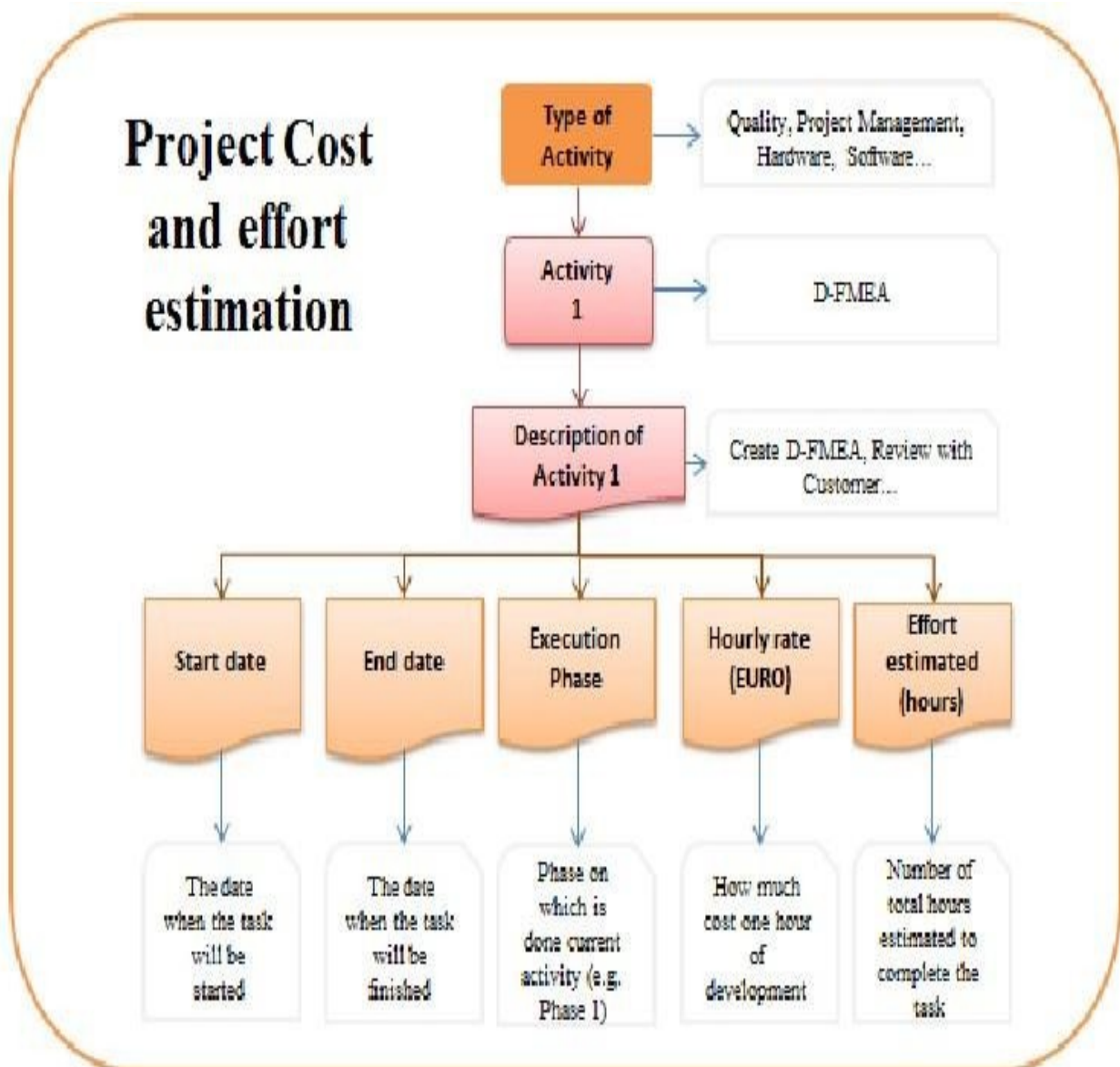




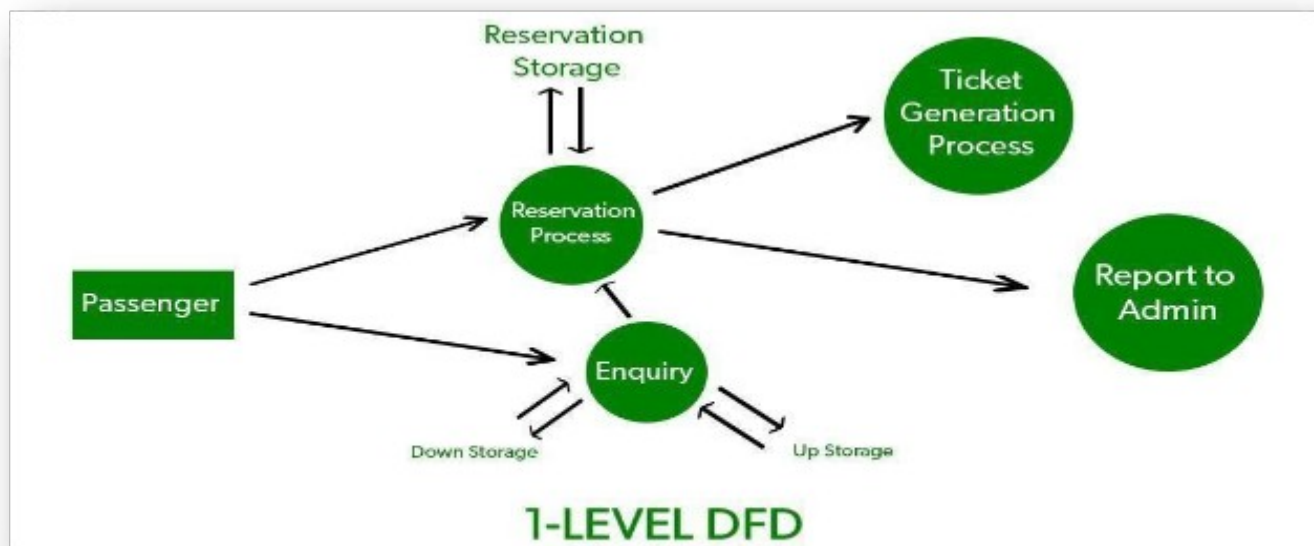
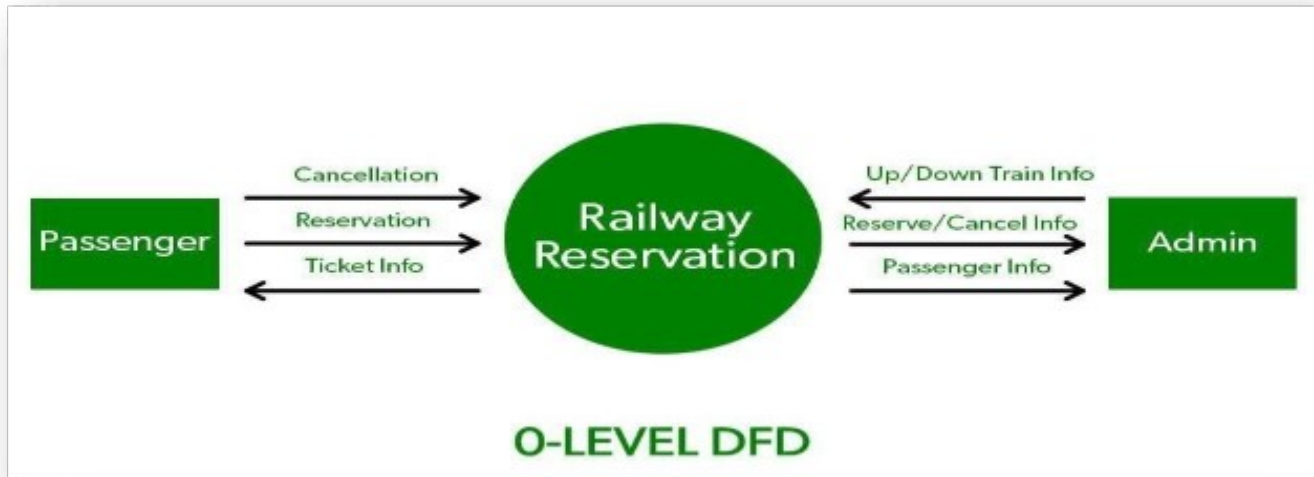
**Practical 05:** Represent project Scheduling of above-mentioned projects.



**Practical 06:** Use any model for estimating the effort, schedule and cost of software project.



**Practical 07: Develop DFD model (level 0, level 1 DFD and Data dictionary) of the project.**



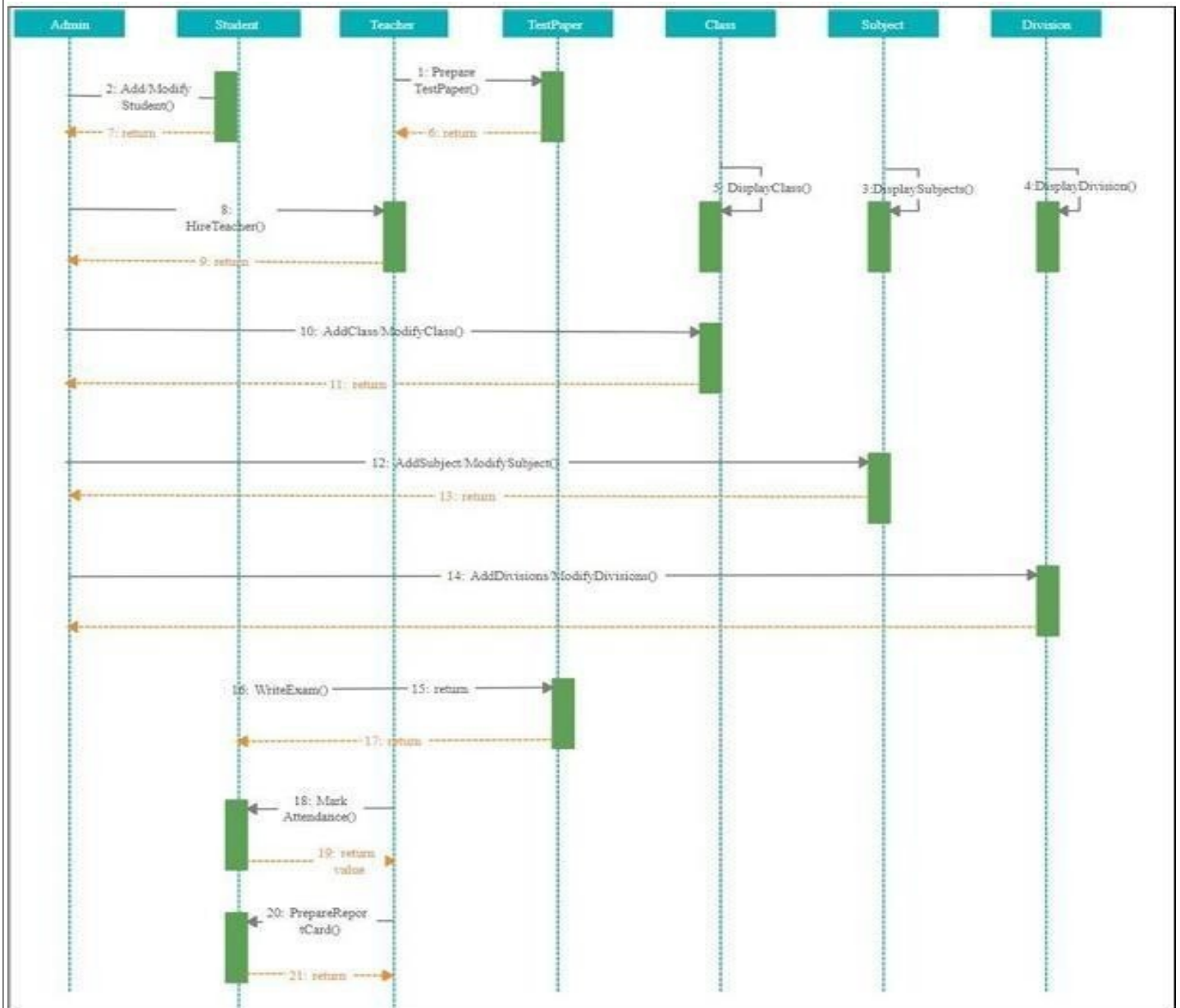
**Data Dictionary Notations Tables:**

The Notations used within the data dictionary are given in the table below as follows:

| Notations   | Meaning                                          |
|-------------|--------------------------------------------------|
| $X = a+b$   | X consists of data elements a and b.             |
| $X = [a/b]$ | X consists of either data element a or b.        |
| $X = a X$   | X consists of optimal data elements a.           |
| $X = y[a]$  | X consists of y or more events of data element a |

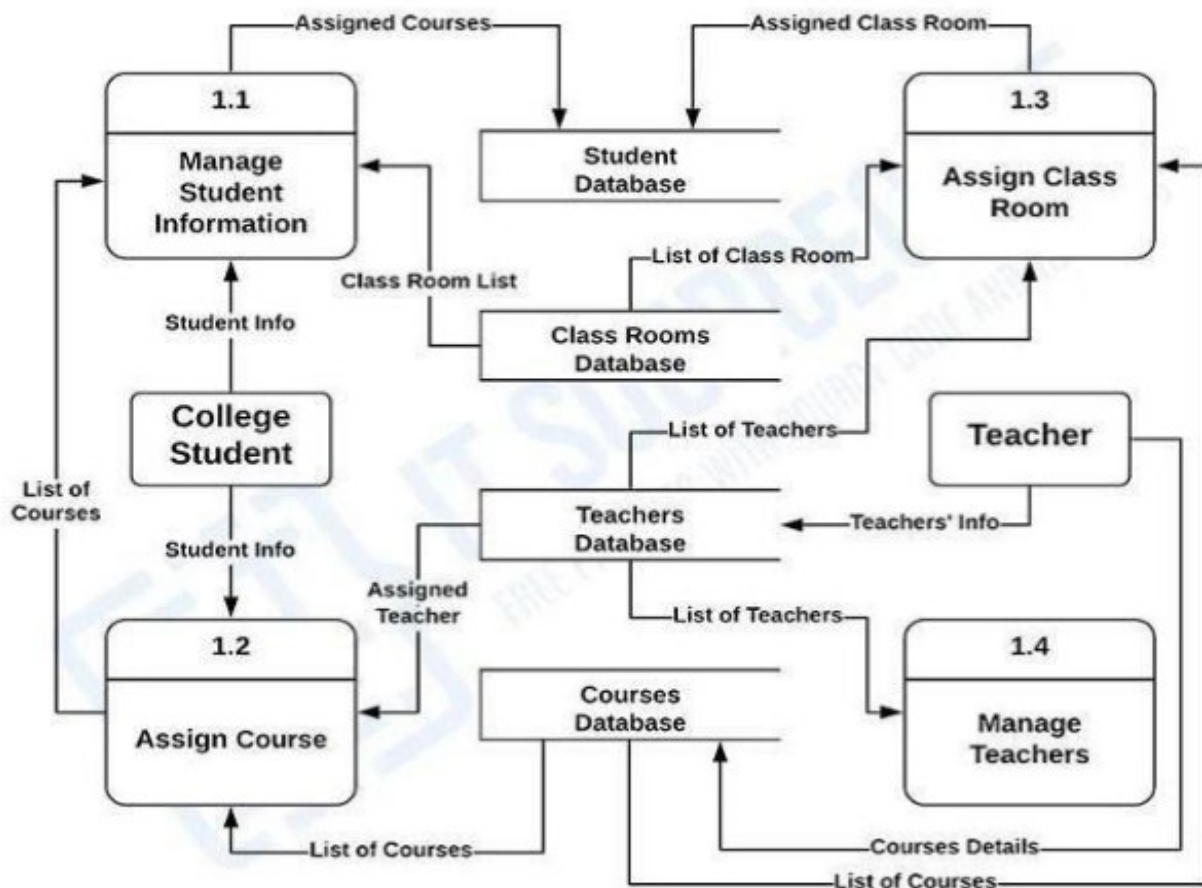
|               |                                                            |
|---------------|------------------------------------------------------------|
| $X = [a] z$   | X consists of z or less events of data element a           |
| $X = y [a] z$ | X consists of some events of data elements between y and z |

## Practical 08: Develop sequence diagram.



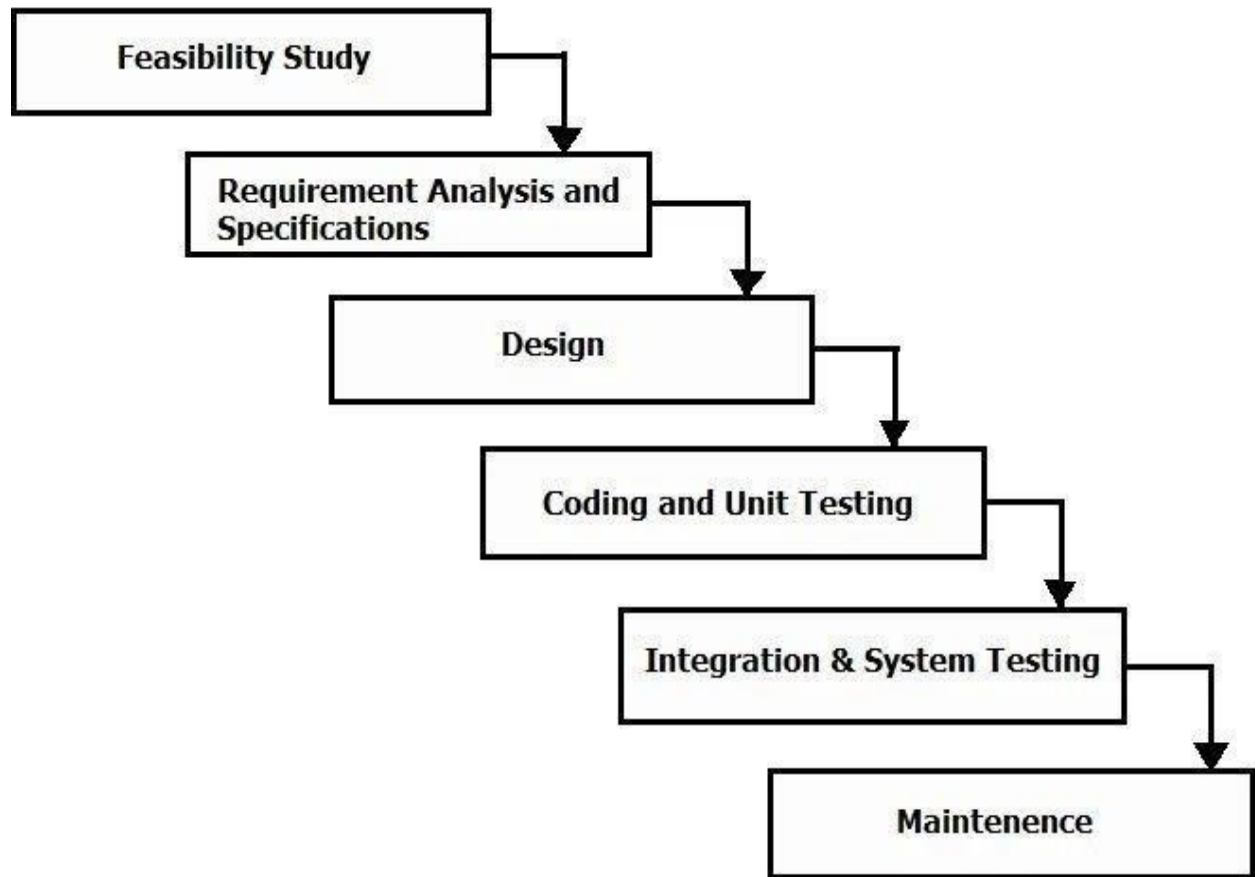
**Practical 09: Develop Structured design for the DFD model developed.**

### College Management System DFD

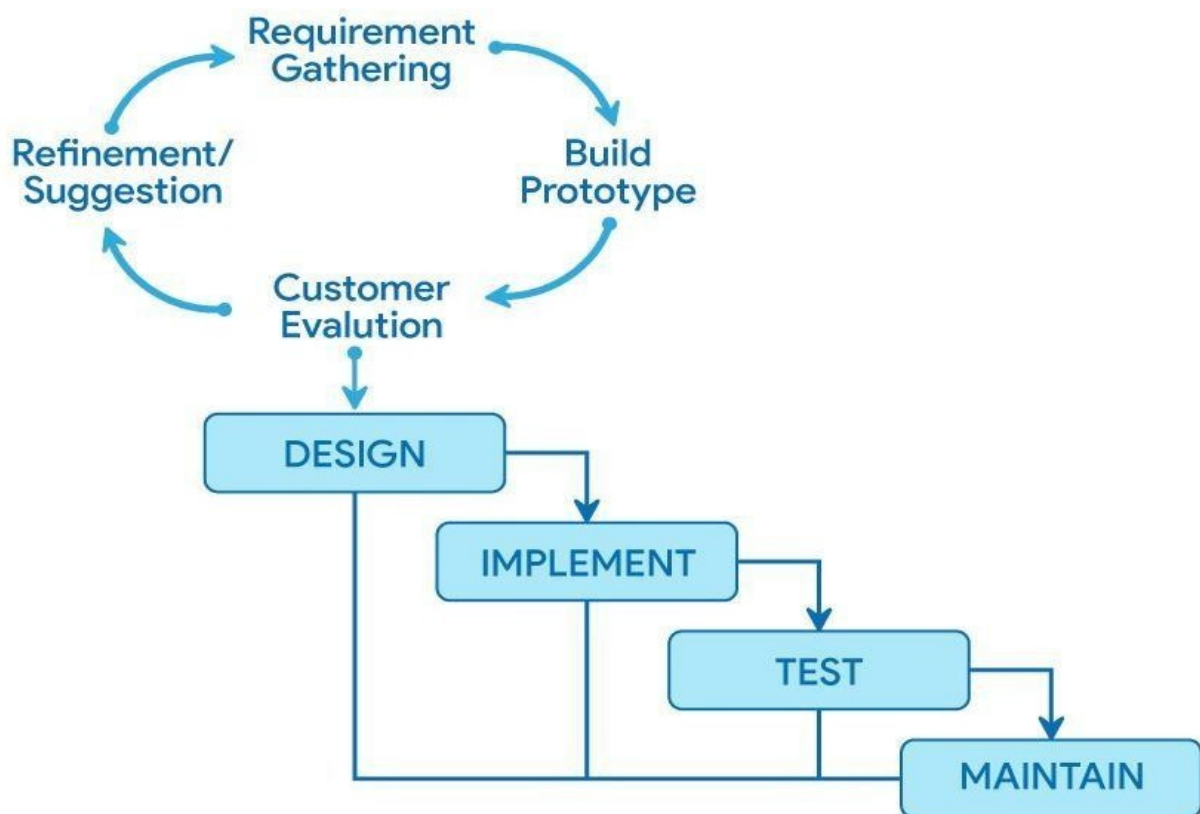


**Practical 10:** Develop the waterfall model, prototype model and spiral model of the product.

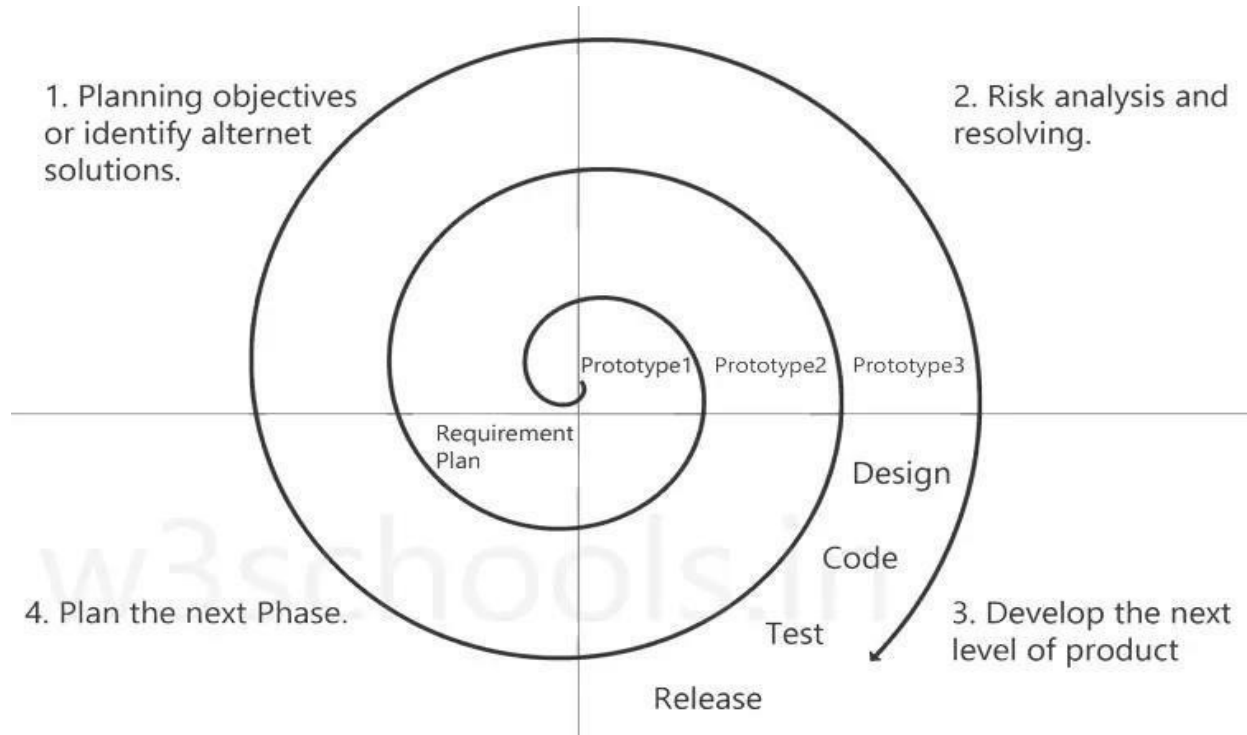
**1). Waterfall Model:**



**2). Prototyping Model:**



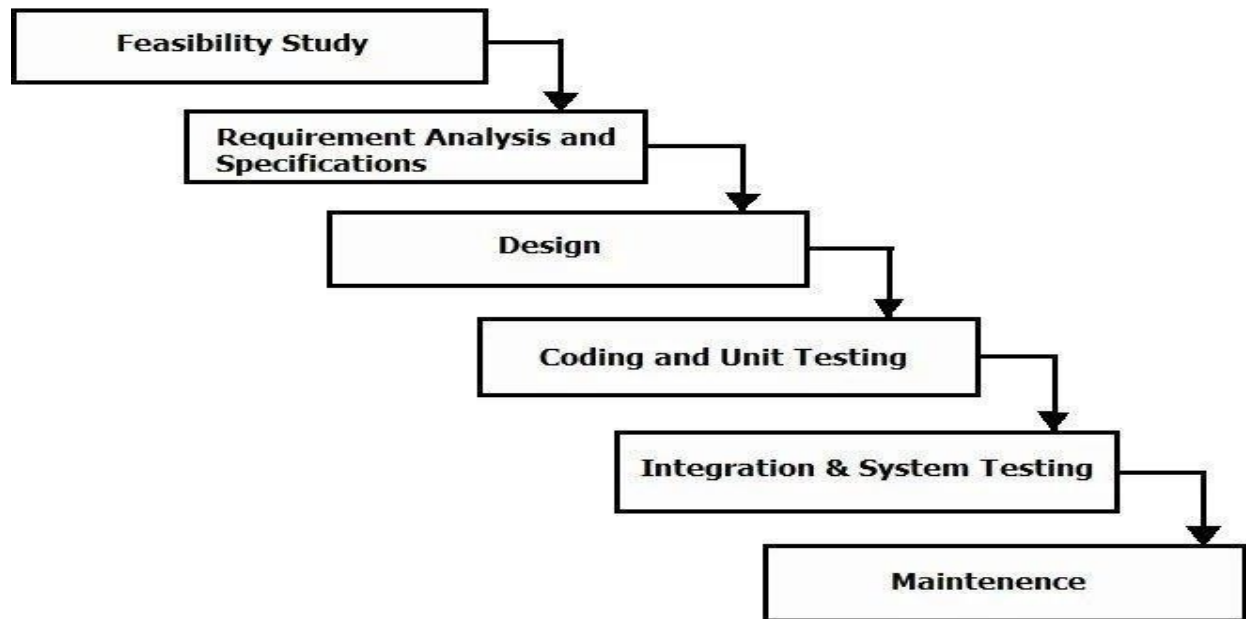
### 3). Spiral Model:





**Practical 11: Explain with reason which model is best suited for the product.**

The Waterfall Model was the first Process Model to be introduced. It is also referred to as a linear-sequential life cycle model. It is very simple to understand and use. In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases.



**Advantages:**

The linear nature of the waterfall development method makes it easy to understand and manage. Projects with clear objectives and stable requirements can best use the waterfall method. Less experienced project managers and project teams, as well as teams whose composition changes frequently, may benefit the most from using the waterfall development methodology.

**Disadvantages:**

The waterfall development method is often slow and costly due to its rigid structure and tight controls. These drawbacks can lead waterfall method users to explore other software development methodologies.

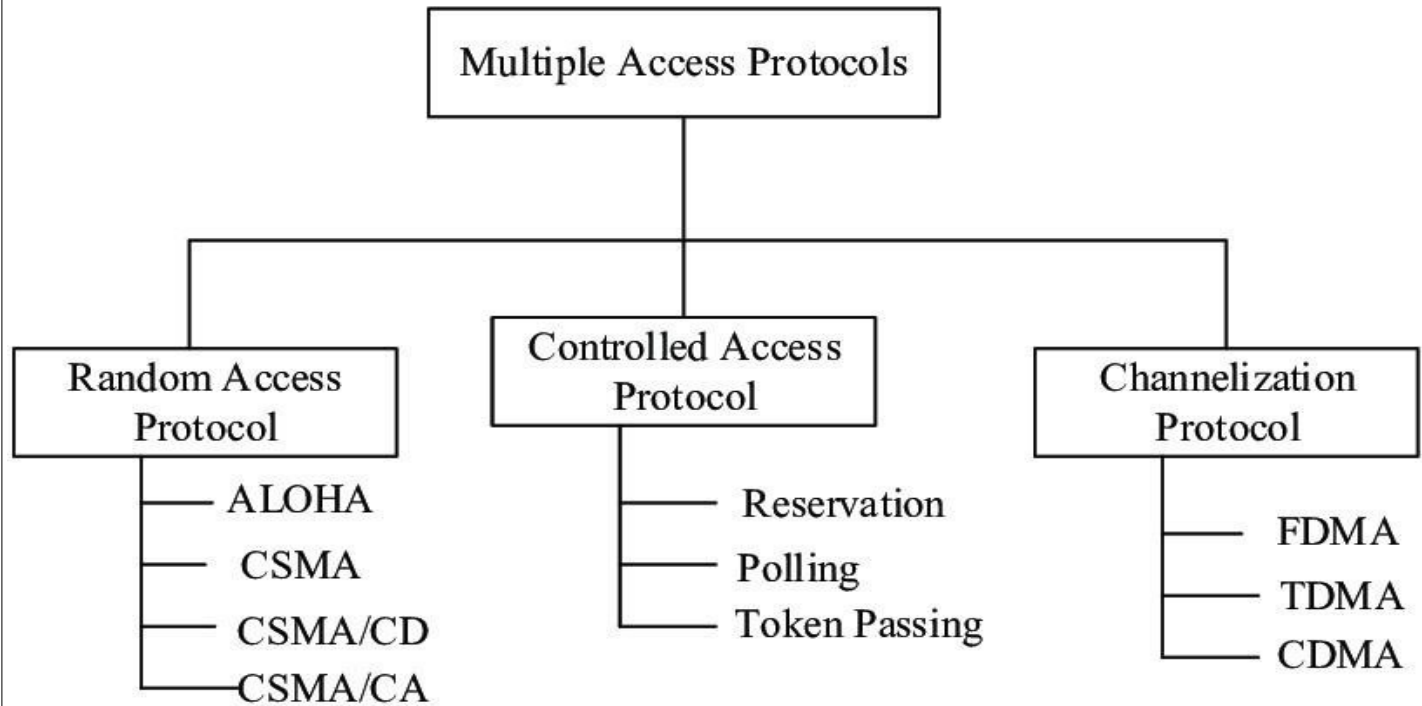
**Practical 12: Develop a working protocol of any of two problems.**

**Data Link Control:**

- This involves framing, error detection, and error correction. It ensures that data packets are formatted properly for transmission and checks for errors during transfer to maintain data integrity.

**Multiple Access Control (MAC):**

- This manages how multiple devices share a single communication channel, preventing collisions and ensuring efficient use of the network. Protocols like CSMA/CD (Carrier Sense Multiple Access with Collision Detection) in Ethernet or CSMA/CA (Collision Avoidance) in Wi-Fi are part of this function.



### **Practical 13: Use LOC, FP and Cyclomatic Complexity Metric of above-mentioned problem.**

#### **1. Lines of Code (LOC):**

- **Purpose: Measure the size of the source code.**
- **Steps:**
  - Write code implementing the functionalities of the Data Link Layer (e.g., framing, error detection, MAC protocols).
  - Count the total lines of code, excluding comments and blank lines.

#### **Example:**

```
python
# Sample LOC example for error detection
def check_errors(data): checksum =
sum(data) % 256 return checksum == 0
```

$\frac{1}{2\pi}$  Count every line in the function above for LOC calculation.

#### **2. Function Points (FP):**

- **Purpose: Measure functionality delivered by the software.**
- **Steps:**
  - Identify all inputs, outputs, and user interactions in the code. o Assign weights based on complexity (simple, average, or complex).
- **Example:**
  - $\frac{1}{2\pi}$  Input: Data packets.
  - $\frac{1}{2\pi}$  Output: ACK/NACK responses.
  - $\frac{1}{2\pi}$  Complexity: Medium complexity functions for error detection and MAC management.

$FP = \text{Sum of (Function Types} \times \text{Weights)}$

#### **3. Cyclomatic Complexity:**

- **Purpose:** Measure the complexity of the code by calculating the number of decision points.
- **Formula:  $M = E - N + 2P$** 
  - Where:**
    - E = Number of edges in the code flow graph.
    - N = Number of nodes.
    - P = Number of connected components.

#### **Steps:**

- Analyze the control flow of the program.
- Create a flow graph based on decision points (like loops, if-else statements).
- Count edges and nodes to calculate MM.

**Example:**

```
def process_packet(packet)
: if packet.is_corrupt():
return "Discard" elif
packet.is_duplicate():
    return "Ignore"
else:
    return "Process"
```

- Decision points: **if, elif, else.**
- Cyclomatic Complexity: 3 (one for each branch).

## Practical 14: Find Maintainability Index and Reusability Index of above-mentioned problem.

### 1. Maintainability Index (MI)

• **Purpose:** Indicates how easy it is to maintain the code.

• **Formula:**

$$MI = 171 - 52 \times \ln(\text{Halstead Volume}) - 0.23 \times \text{Cyclomatic Complexity} - 16.2 \times \ln(\text{LOC})$$

**Where:**

- o **Halstead Volume** = Measures complexity based on operators and operands in the code.
- o **Cyclomatic Complexity** = Measures decision points in the code.
- o **LOC (Lines of Code)** = Total number of lines in the code.

Steps:

1. Calculate Halstead Volume using code tokens (operators and operands).
2. Insert values into the formula for MI.
3. Interpret the result:
  - a. **Higher MI (>85):** High maintainability.
  - b. **Moderate MI (50–85):** Average maintainability.
  - c. **Low MI (<50):** Poor maintainability.

### 2. Reusability Index (RI)

• **Purpose:** Measures how easily code can be reused in other projects.

Factors to Consider:

- o **Modularity:** Divide the code into self-contained functions or classes.
- o **Code Dependencies:** Reduce hardcoded dependencies on external files.
- o **Code Comments and Documentation:** Improve clarity for others to understand.
- o **Parameters and Generality:** Use adaptable logic instead of case-specific solutions.

**Example:**

- Assess modular functions such as framing, error detection, or MAC protocols.
- Code with reusable logic (e.g., checksum-based error detection) receives a higher RI.

**Note:** Unlike MI, RI does not have a fixed formula. It is typically evaluated qualitatively or with custom metrics based on modularity, coupling, and clarity.

## **Practical 15: Using any Case Tool find number of statements, depth and complexity of the prototype.**

### **1. Visual Paradigm**

- Good for UML diagrams, modeling, and reverse engineering.
- Can analyze source code to estimate complexity.

### **2. Enterprise Architect**

- Provides detailed analysis and generates metrics like cyclomatic complexity.
- Can be used to visualize class diagrams, sequence diagrams, and analyze complexity.

### **3. SonarQube**

- Great for code quality analysis and measuring cyclomatic complexity.
- Provides detailed reports for large codebases.

### **4. CAST Software**

- Deeply analyzes code complexity and software architecture.
- Provides visual insights into complexity and dependencies.

---

## **Steps to Analyze Prototype Using a CASE Tool**

### **Step 1: Import or Load Prototype Code**

- Open the chosen CASE tool.
- Import the prototype code (Java, C++, Python, etc.) or the design/model.
- If using Visual Paradigm or Enterprise Architect, load the UML model or source code.

### **Step 2: Generate Code/Diagram Analysis**

- Perform a reverse engineering process to extract design and logic.
- For source code, look for the feature that analyzes cyclomatic complexity.

### **Step 3: Analyze Metrics**

#### **1. Number of Statements**

- a. Count the number of statements (Lines of Code).
- b. Most tools automatically generate this or display LOC (Lines of Code).

#### **2. Depth of Nesting**

- a. Identify the highest level of nesting (loops, conditions, etc.).
- b. Tools like SonarQube highlight deeply nested methods/functions.

### 3. Cyclomatic Complexity (McCabe's Complexity)

a. Measures the number of independent paths through the code.

b. Formula:

$$V(G) = E - N + 2P$$

Where:

- **E** = Number of edges in the control flow graph
- **N** = Number of nodes
- **P** = Number of connected components (usually 1 for a single program)

#### Example Cyclomatic Complexity Values

- **1–10** → Low complexity, easy to maintain.
- **11–20** → Moderate complexity.
- **21–50** → High complexity, prone to errors.
- **> 50** → Very complex, difficult to manage.